RIC Architecture & RIC Security

Supervisor: Dr. Chatura Seneviratne
Co-Supervisor 1: Prof. Dr. An Braeken
Co-Supervisor 2: Mr. Pramitha Fernando

EG/2021/4474 Dewmith M.K.J.
EG/2021/4485 Dilshan M.D.K.
EG/2021/4807 Sewinda L.L.D.
EG/2021/4832 Thilohith T.K.

08/12/2025

CONTENT

01 | Near-RT RIC: Architecture & Foundations

02 | xApp Lifecycle Management (LCM)

03 | Security Threat Landscape for xApps

04 | Threat Containment & Mitigation Strategies.

PART 01

Near-RT RIC: Architecture & Foundations

Near-RT RIC: Role & Architecture Overview

Positioning: Logically situated between the Non-RT RIC and E2 Nodes.

Core Function: Connects to one Non-RT RIC for policy guidance and one or more E2 Nodes for real-time control.

Architecture Components:

- **Near-RT RIC Platform:** Provides common infrastructure (Messaging, SDL, Security, Management) to support applications.
- **xApps:** Independent applications that leverage the platform to control RAN functions.

Key Goal: Enables an open, multi-vendor environment for optimizing RAN resources in near-real-time (10ms to 1s loop).

Platform Foundations: Messaging, APIs & Security

Messaging Infrastructure: Provides low-latency delivery between internal endpoints. Supports Registration/Discovery of endpoints and multiple modes like Point-to-Point and Publish/Subscribe.

API Enablement:

- Acts as a registry for Near-RT RIC APIs, enabling xApps to register their services and discover others.
- Handles authentication to ensure xApps only access authorized services.

Conflict Mitigation: Detects and resolves overlapping requests from different xApps to prevent network instability.

Security: Prevents malicious xApps from abusing radio information or unauthorized control.

The Data Layer: SDL, R-NIB & UE-NIB

Shared Data Layer (SDL):

- **Purpose:** Decouples xApps from the underlying database technology, providing a high-level API to read/write/subscribe to data.
- **Function:** Enables xApps to receive notifications when network information changes

Radio-Network Information Base (R-NIB):

- Maintains configurations and near-real-time state of connected E2 Nodes and the mappings between nodes and cells.

UE-Network Information Base (UE-NIB):

- Maintains a list of **UEs** (User Equipment), tracking their identities and correlating them across E2 Nodes.

Primary Interfaces: A1 (Policy) & E2 (Control)

A1 Interface (Northbound to Non-RT RIC):

- **Policy Guidance**: Receives high-level policies (A1 Policies) to guide xApp behavior.
- **Enrichment Information**: Receives external context data to assist optimization.

E2 Interface (Southbound to E2 Nodes):

- **Control & Data**: Carries "REPORT", "INSERT", "CONTROL", and "POLICY" services.
- **E2 Termination**: Handles SCTP connections, decodes ASN.1 messages, and routes data between E2 Nodes and xApps.

Management & Exposure: O1 & Y1

O1 Interface (Management to SMO):

- **FCAPS:** Handles Faults, Configuration, Accounting, Performance, and Security management.
- **Lifecycle:** Manages xApp onboarding, starting, and stopping.

Y1 Interface (Analytics Exposure):

- **Purpose**: Exposes RAN Analytics Information (RAI) generated by the RIC to external "Y1 Consumers".
- **Value:** Allows third-party applications to subscribe to and benefit from the network intelligence produced by xApps.

PART 02

xApp Lifecycle Management (LCM)

xApp Lifecycle Management (LCM)

- xApps are microservice-based applications that provide RAN functionalities and optimization.
- Its lifecycle is orchestrated **by the SMO** and **executed on O-Cloud** infrastructure.

Lifecycle Management (LCM) is managed by the SMO via O1 towards the Near-RT RIC, while xApps interact with the Near-RT RIC platform via APIs.

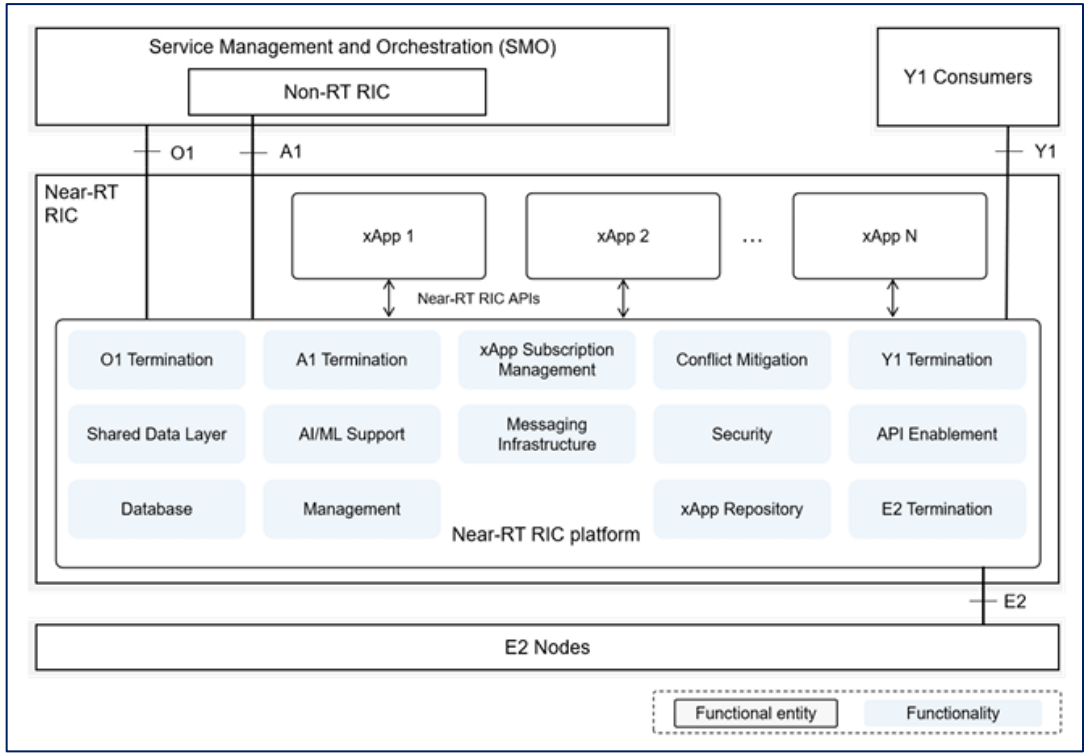


Figure1.1: Near-RT RIC Architecture view [1]

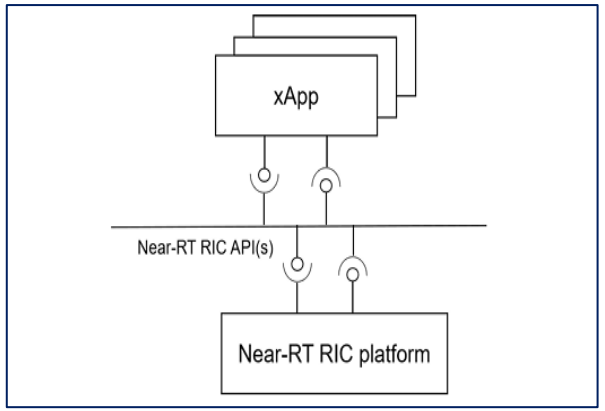


Figure1.2: Service-based representation for internal Near-RT RIC [1]

1.1 Secure Onboarding

The Process begins with onboarding is the contract and safety inspection

- Contract → trust, signing, verification
- Safety inspection → integrity and compliance

- **Process:** The Solution Provider signs the xApp package. Then the Service Provider validates the xApp package integrity and may re-sign it according to operator security policy
- The package is published to a catalogue for SMO visibility.
- Digital signatures ensure the app is from a valid provider and is integrity protected.

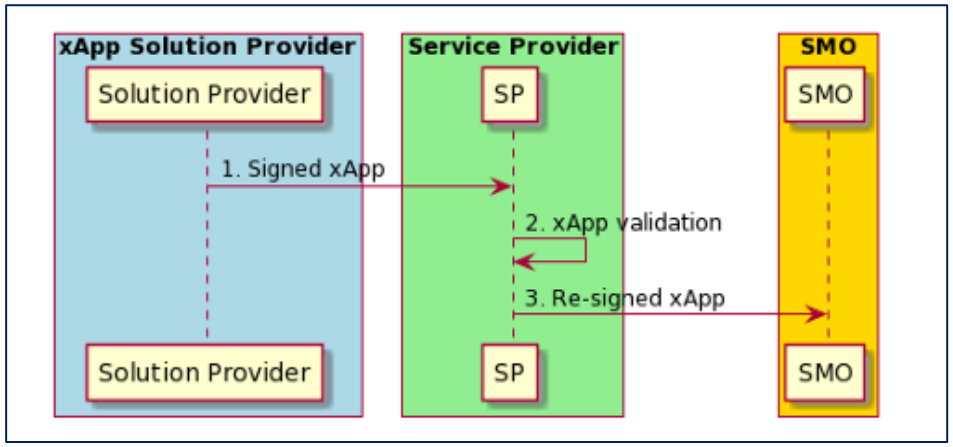


Figure2: Onboarding xApps to SMO [2]

1.2 Deployment & Resource Allocation

During deployment, the SMO instructs the O-Cloud DMS to deploy the xApp and allocate required resources.

- **Action:** The SMO maps deployment parameters and configuration values and instructs the O-Cloud Deployment Management Service (DMS) to instantiate the xApp.
- **Infrastructure:** The DMS allocates CPU, memory, and I/O resources on the O-Cloud.
- **Tracking:** A Deployment ID is assigned, which the SMO uses to update its inventory.

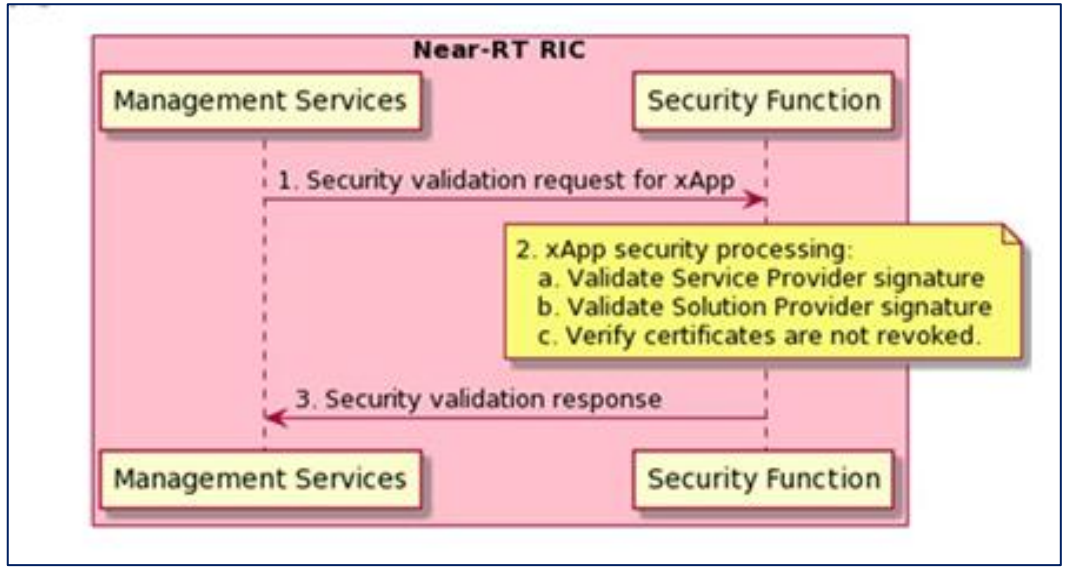


Figure3: Authentication during Development Process [2]

2.1 xApp Registration & Identity

Once initialized on the O-Cloud, the xApp must identify itself to the platform via the xApp Registration procedure. The xApp sends a registration request to the Near-RT RIC Management Services, which authenticates the xApp and validates its capacity to run on the platform

- **Identity:** The Near-RT RIC platform assigns a unique xApp ID in UUID version 4 format.
- **Validation**: The platform authenticates the xApp instance and validates its identity and permissions.
- **Certificates:** The platform may act as a Registration Authority to fetch an operator certificate (SVID) for the xApp

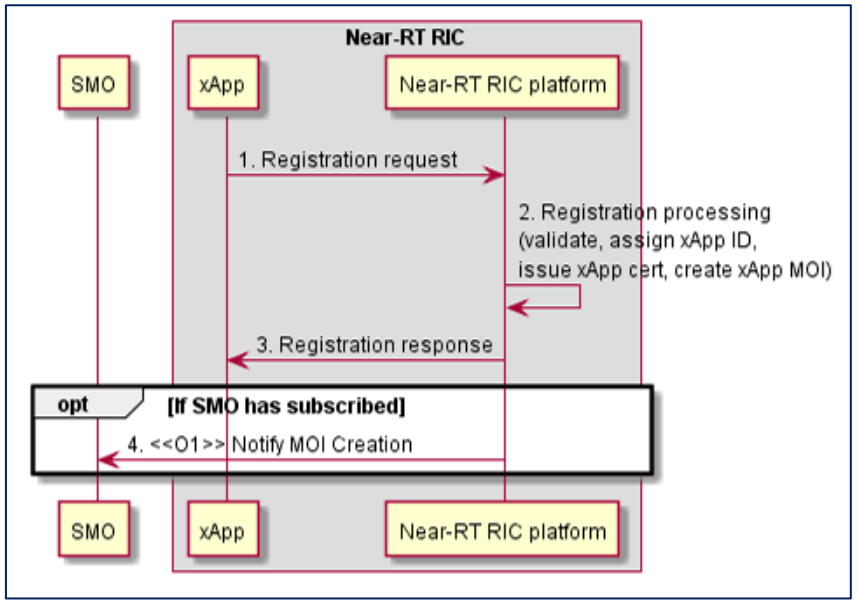


Figure4: xApps registration Procedure [1]

2.2 Service & Node Discovery

The xApp uses **Enablement APIs** to discover available Near-RT RIC platform services

- **RAN Awareness:** xApps discover available E2 nodes and their capabilities via Enablement APIs and platform data services (e.g., SDL).
- **Dynamic Updates:** xApps subscribe to SDL notifications to track changes in E2 node states (e.g., "Connected" or "Reset")

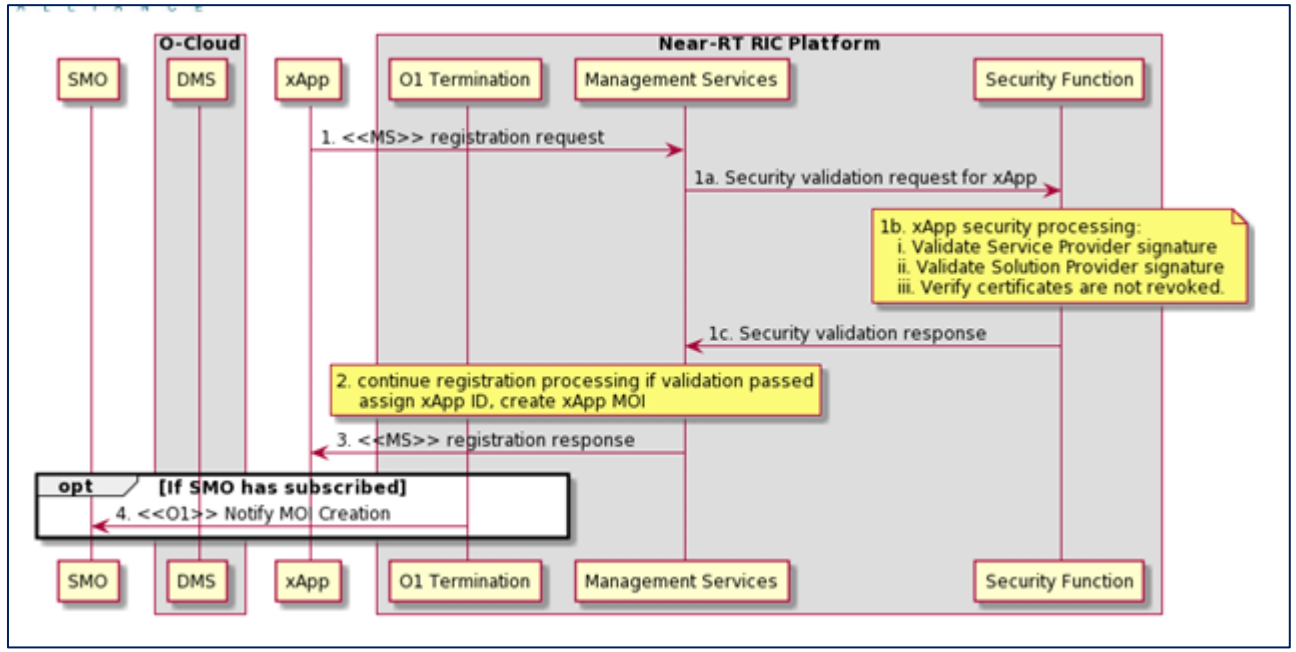


Figure5: Extended xApps registration Procedures [2]

3.1 Configuration & MOI Initialization

Initialization via the O1 interface creates a Managed Object Instance (MOI). After registration, the Near-RT RIC platform creates the xApp MOI with its ID and parameters, sending a Configuration Request to synchronize settings.

- **MOI Creation**: The platform creates a Managed Object Instance (MOI) in the Configuration Database to represent the xApp.
- **Synchronization:** The platform sends a Configuration Request to the xApp via the Management API to ensure internal settings match the MOI.
- **Flexibility:** xApps are configurable based on parameters received from the Near-RT RIC platform, which is managed by the SMO over O1.

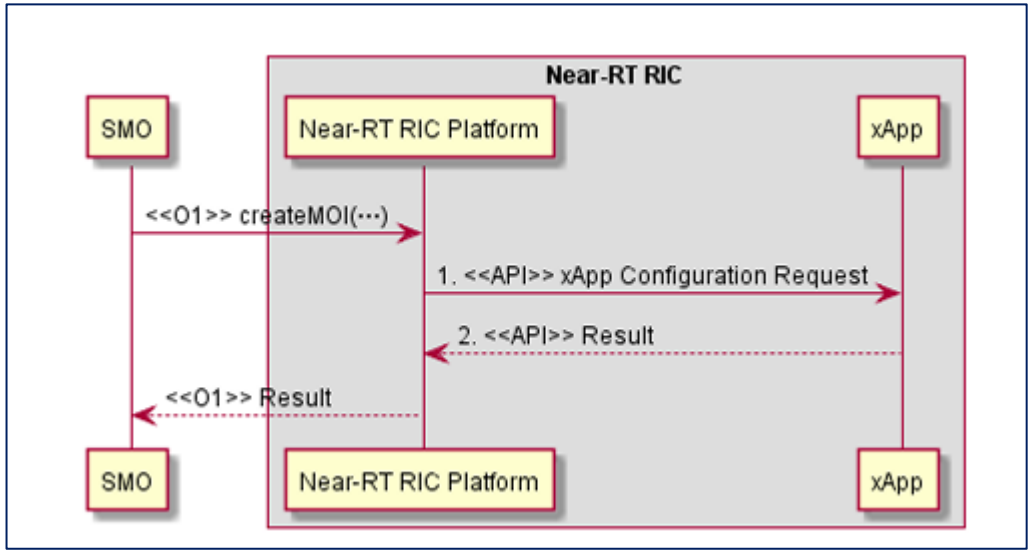


Figure6: xApp Create MOI [1]

5. Termination & Cleanup

Termination begins when the SMO sends an edit-config delete request over O1, prompting the platform to remove the xApp's MOI. Cleanup ensures no leftover processes or database entries remain, preventing "ghost" artifacts.

- **SDL Deregistration**: The xApp sends an **SDL Client Deregistration** request to release its connection and permissions to the database
- **Resource Release:** The platform updates subscription records and removes the xApp from all distribution lists
- **Data Cleanup**: **E2NodeInfo** records are deleted after an E2 node is disconnected, once sufficient time is allowed for all associated xApps to complete their independent cleanup mechanisms

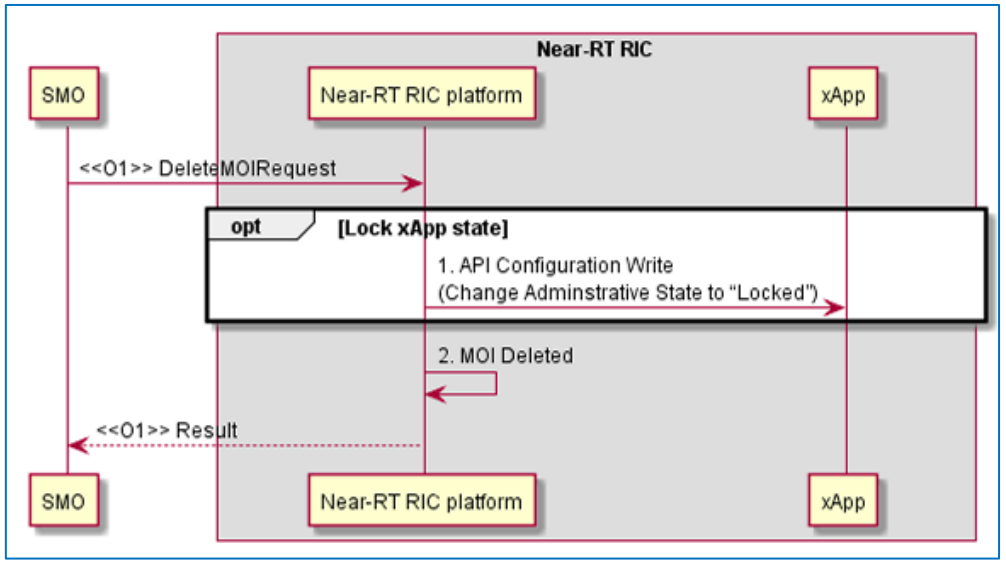


Figure8: xApp Delete MOI [1]

4.1 Operational Management

➢ During the operational phase, the xApp is managed through several mechanisms, here some,

- **Policy Updates (A1):** The xApp receives policy-driven guidance from the Non-RT RIC via the A1 interface. The platform routes these policy management messages to the specific xApp based on the A1 policy type.

- **Performance Monitoring (PM):** The platform reports PM measurements associated with individual xApps over the O1 interface. xApps notify the platform when PM data files are ready or stream data directly to authorized consumers (SMO).

- **State Management:** The platform can manage the Administrative State (e.g., locking/unlocking) of an xApp via O1 commands.

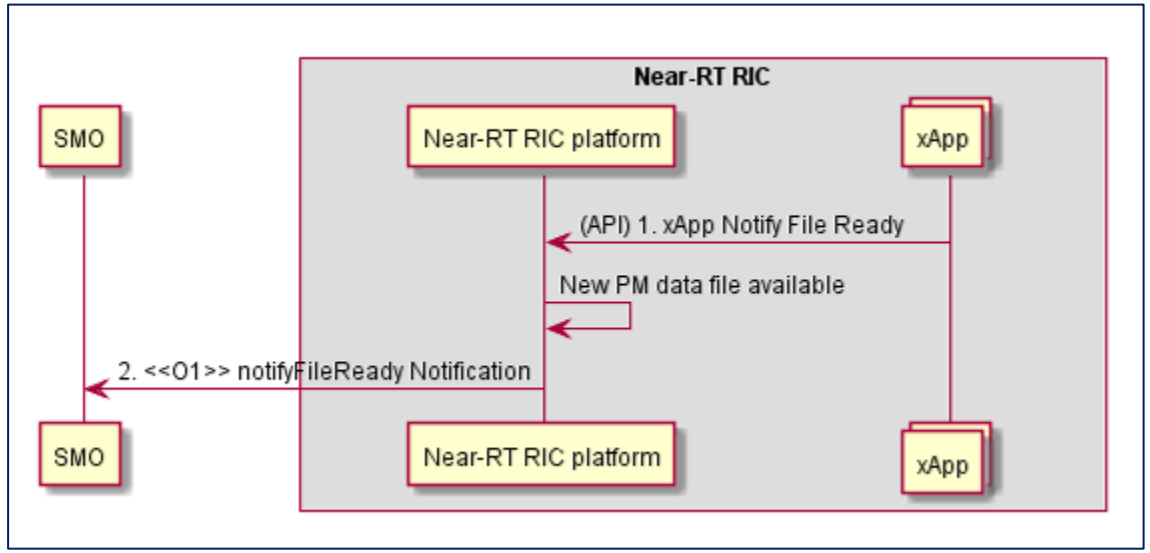


Figure7: Performance Data File Reporting [1]

Summary of the xApp Journey

The xApp is like hiring manpower at a firm for a specific duration

1. **Onboarding** is the background check and contract signing with the SMO.
2. **Registration** is when they report to operate a machine to get their unique ID badge (xApp ID) for access.
3. **Discovery** is checking the office map to find where the available machine operations/tasks are.
4. **Configuration** is being assigned a specific machine with credentials/permissions.
5. **Operations** are the daily work guided by company policies (A1), with regular status reports sent to the boss (O1).
6. **Termination** is handing their presence and cancels their access to ensure no digital "ghosts" remain in the system.

- **Lifecycle Integrity**: From signing during onboarding to resource release during termination, every step is validated by the SMO and RIC platform to maintain network stability.

- **Zero Trust**: Operations are built on zero-trust principles, requiring mutual authentication and authorization for all data access

PART 03

Security Threat Landscape for xApps

Security Threat Landscape for xApps

- xApps run inside the Near-RT RIC platform and have **near real-time control over RAN functions.**
- A security breach in an xApp can:
  - Affect multiple network services
  - Impact other xApps sharing the same platform
  - xApps are critical attack surface in Open RAN.

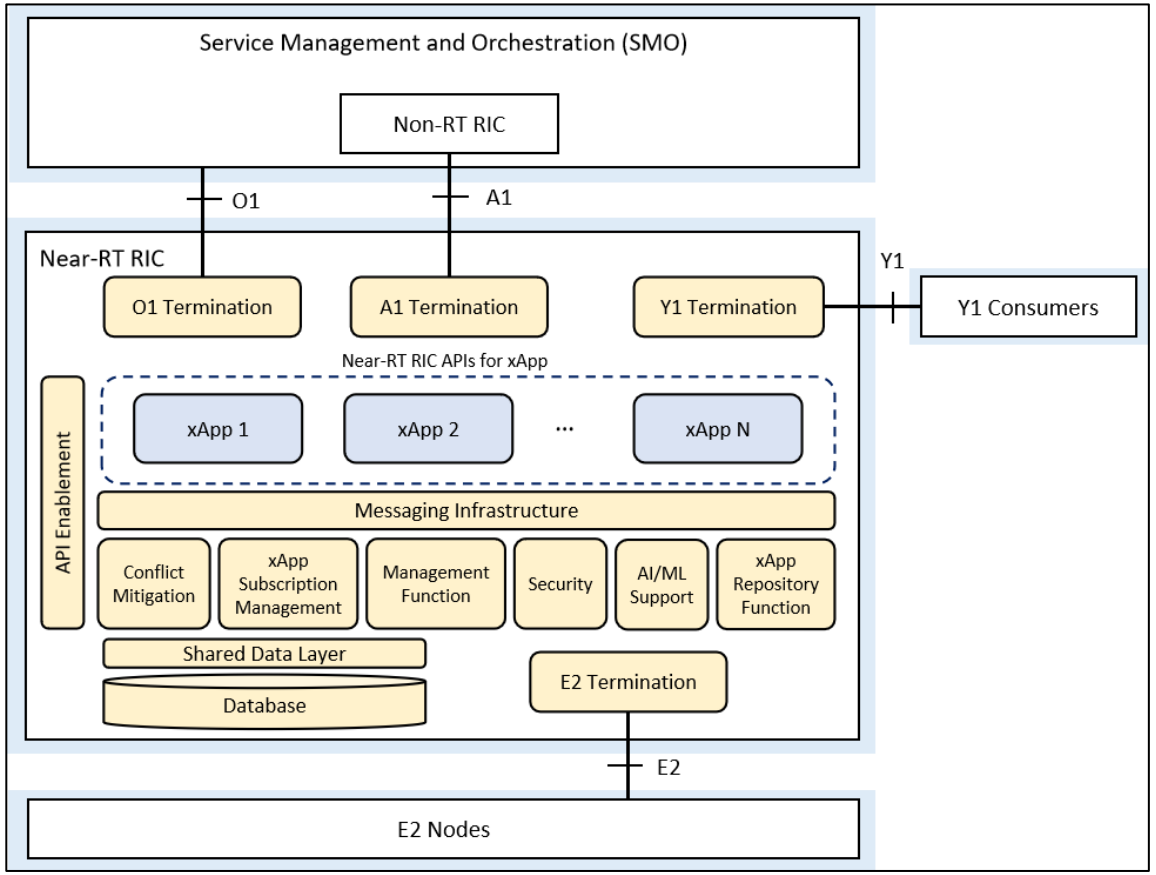


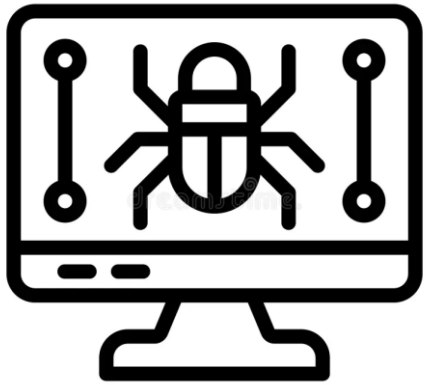
Figure 8: Near-RT RIC Internal Architecture

Why xApp Security Is Critical

- xApps:
  - Are often third-party developed
  - Use open-source libraries
  - Share platform resources (CPU, memory, SDL)
- Trusting all xApps by default is unsafe
- xApps can be compromised

xApp Threat Model

Threat Type	Description
Malicious xApp	xApp designed with harmful intent from deployment
Compromised xApp	Legitimate xApp taken over via vulnerabilities or attacks



- Compromised xApps are:
  - Harder to detect
  - More realistic in operational networks

Threat Causes for Compromised xApps

- Security misconfigurations
  - Enabled but unnecessary services
- Weak authentication and authorization
- Vulnerabilities in open-source libraries
- Supply-chain attacks during xApp packaging
- Backdoors inserted intentionally or unintentionally
- Injection attacks leading to remote code execution

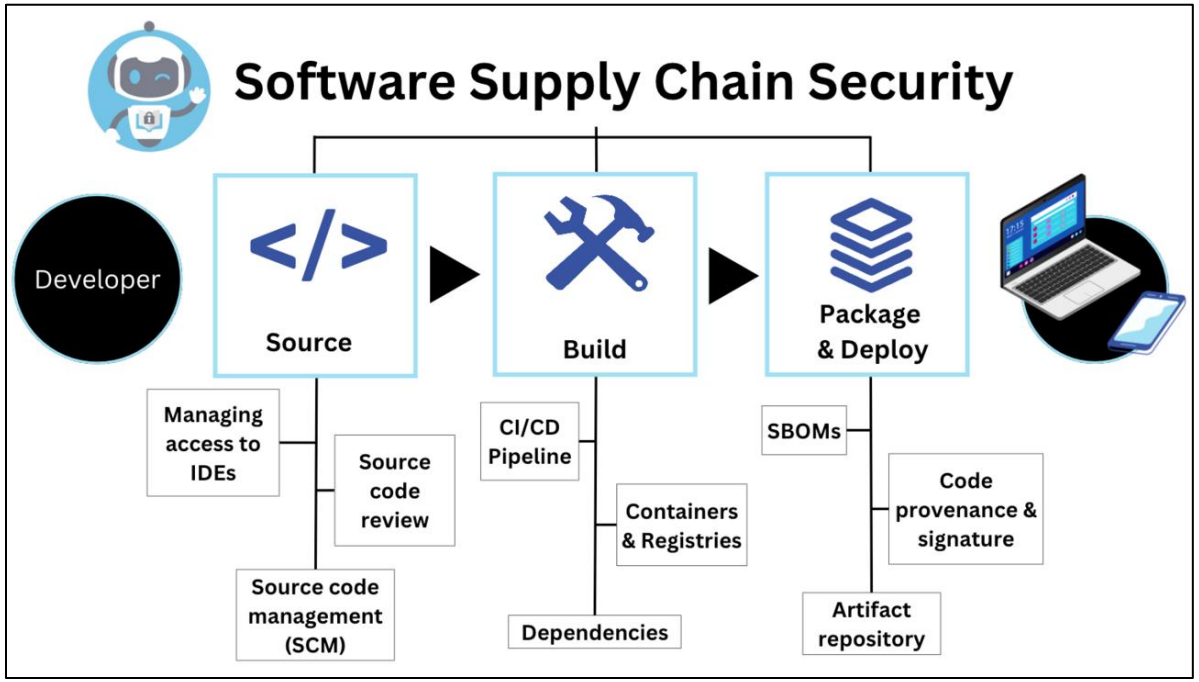


Figure 9 : Software Supply Chain

Key Security Issue 1: Resource Exhaustion

- A compromised xApp may:
  - Consume excessive CPU or memory
  - Flood APIs with requests

- Since Near-RT RIC is a shared platform, this affects all xApps

Impact	Result
CPU/Memory exhaustion	Near-RT RIC slowdown
API flooding	Denial of Service (DoS)
Shared resources	Other xApps fail

➢ **Impact of Resource Exhaustion**

- Near-RT RIC fails to meet real-time deadlines
- Legitimate xApps cannot execute control loops
- Network performance degrades
- Considered a high-impact threat

Key Security Issue 2: Unauthorized Data Access

- xApps access network data via the Shared Data Layer (SDL)

- Sensitive data stored in:
  - UE-NIB – UE identity and mobility information
  - R-NIB – RAN configuration and state

Data Source	Risk
UE-NIB	Privacy leakage
R-NIB	Network intelligence exposure

➢ **Impact of Unauthorized Data Access**

- Leakage of private user information
- Exposure of operator network topology
- Enables further targeted attacks
- Violates privacy and regulatory requirements

Key Security Issue 3: E2 and A1 Message Manipulation

➢ E2 Message Manipulation

- Compromised xApp can send malicious control commands via E2 interface
- These commands directly affect RAN behavior

- **Examples:**
  - Incorrect power control
  - Unnecessary handovers
  - Biased scheduling decisions

➢ A1 Policy Manipulation

- xApps receive policy guidance via A1 interface
- Compromised xApp may:
  - Ignore policies
  - Misuse policy parameters
- Leads to misaligned network optimization decisions

Figure 10 : Manipulation of E2 control and A1 policy messages by a compromised xApp

Key Security Issue 4 – Conflict Exploitation Between xApps

- Multiple xApps may control overlapping RAN functions
- Compromised xApp can intentionally:
  - Issue contradictory control actions

➢ **Example:**
- One xApp increases transmit power
- Another xApp decreases transmit power

➢ **Results in:**
- Control-loop oscillations (ping-pong effect)
- QoS and QoE degradation

➢ **Impact of xApp Conflicts**

Issue	Effect
Control oscillations	Network instability
Conflicting RRM decisions	QoS degradation
Multiple xApps	Increased attack surface

- Conflicts may be **accidental or malicious**

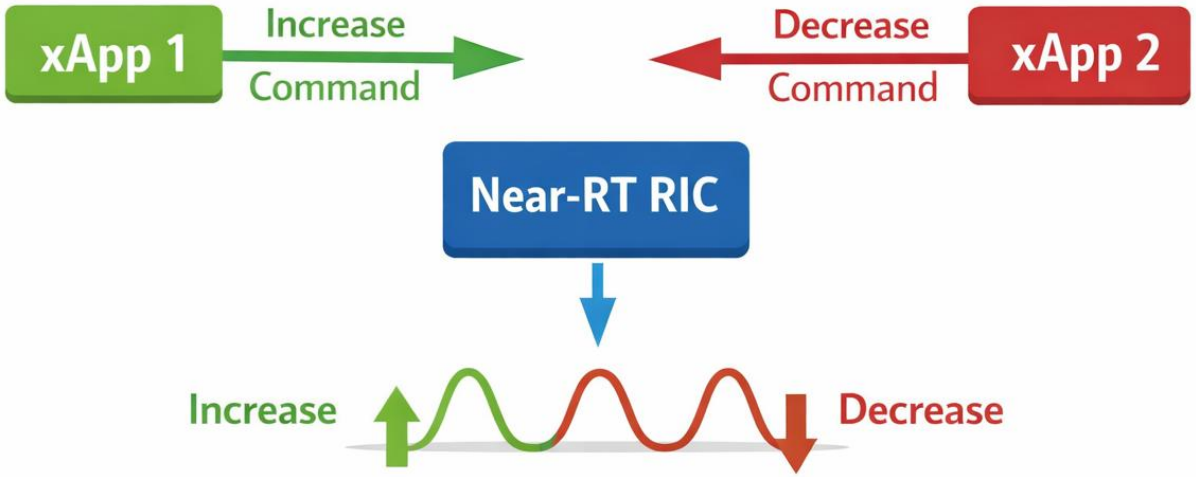


Figure 11 : Conflicting xApps causing oscillation in RAN performance

Summary of xApp Security Threats

Threat Category	Impact
Resource Exhaustion	RIC availability loss
Unauthorized Data Access	Privacy violation breach
Message Manipulation	RAN instability
Conflict Exploitation	Performance degradation

- **This Highlights the need for:**
  - Platform-level security controls
  - Strong isolation and monitoring mechanisms

Why Platform-Level Security Is Required

- xApps cannot be fully trusted
- Security must be enforced by:
  - Near-RT RIC platform
  - SMO and O-Cloud
- Leads to the need for:
  - Isolation
  - Authentication
  - Conflict mitigation

PART 04

Threat Containment Strategies

Threat Containment and Mitigation

A secure Near-RT RIC platform cannot rely on a single defense mechanism. It requires a layered strategy encompassing strict identity verification, resource isolation, granular access control and intelligent conflict resolution. The following diagram outlines these critical security functions.

Mitigation Strategies for Compromised xApps

- **The Problem:** Legitimate xApps may contain vulnerabilities (misconfigurations, backdoors or injection flaws) that allow attackers to disrupt services or steal sensitive data.
- **The Objective:** Implement a multi-layered containment strategy to prevent a single compromised app from taking over the Near-RT RIC platform or another xApp.
- **The Framework:** Focuses on 4 primary solutions mapped to Key Issue #2 (Compromised xApps).
  - Solution #1: Secure Onboarding & Deployment
  - Solution #2: Open-Source Risk Management
  - Solution #3: API Authentication Schema
  - Solution #4: OAuth 2.0 Authorization Framework

Solution #1: Secure Onboarding & Deployment

Lifecycle Phase: Pre-deployment (Onboarding to SMO) and Initialization (Registration).

Implementation:

- **Dual-Signature Process:** The xApp Solution Provider digitally signs the package; the Service Provider performs security/operational testing and adds a second signature.
- **Registration Validation:** During registration, the Management Services send a validation request to the Security Function.
- **Trust Verification:** The Security Function validates both signatures and confirms that certificates are not revoked.

Containment Goal: Ensures xApp code integrity and proves the app came from a trusted provider without tampering.

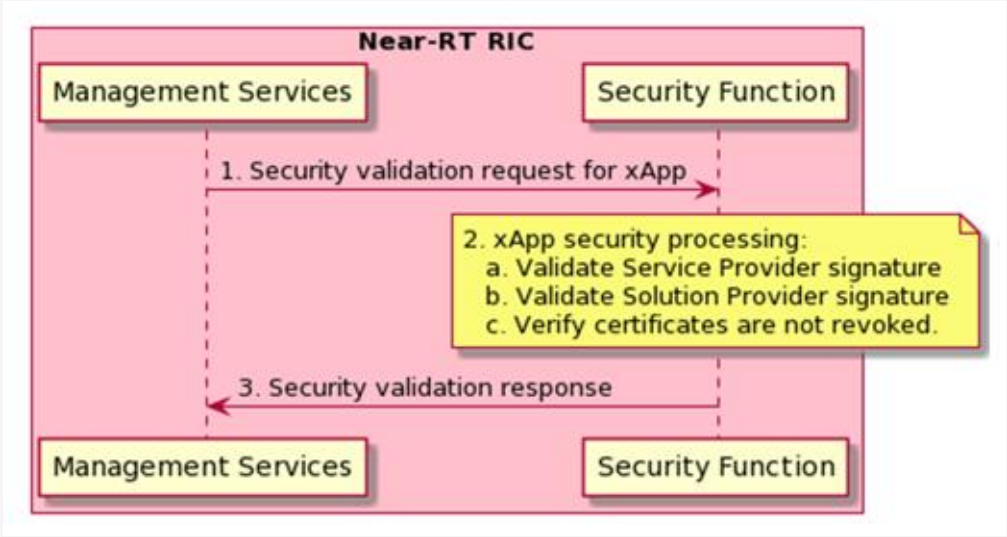


Figure: Authentication during Deployment Process [2]

Solution #2: Security Management of Risks in Open-Source Components

Lifecycle Phase: Continuous Analysis and Build Phase

Implementation:

- **SBOM (Software Bill of Materials):** Use Software Composition Analysis (SCA) tools to identify all open-source libraries and package dependencies.
- **Automated Scanning:** Integrate Static (SAST) and Dynamic (DAST) testing into the CI/CD pipeline.
- **Vulnerability Remediation:** Patch critical publicly known vulnerabilities within 15 days of a fix being available.

Containment Goal: Minimizes the attack surface by ensuring xApps are not running vulnerable third-party code.

Solution #3: Authentication Schema for APIs

Lifecycle Phase: Operational Runtime

Implementation:

- **Mutual TLS (mTLS):** Enforce mTLS for transactional APIs (REST and gPRC) where both the client (xApp) and server (Platform) present X.509 certificates.
- **Application Layer Auth:** Complement mTLS with signed JSON Web Tokens (JWT) attached to service requests.
- **Time Critical APIs:** Use IPSec with IKEv2 certificate-based authentication for time-critical E2-related APIs.

Containment Goal: Prevents unauthorized xApps from masquerading as legitimate ones to request services or leak data.

Solution #4: OAuth Authorization Framework

Lifecycle Phase: Operational Runtime

Implementation:

- **Framework Roles:** The Security Function acts as the Authorization Server, issuing tokens to the xApp.
- **Client Credentials Grant:** xApps obtain access tokens to access specific platform services.
- **Granular Scopes:** Scopes are used to restrict an xApp's access to only the specific APIs and data (like SDL entries) it needs.

Containment Goal: Implements the Principle of Least Privilege, ensuring a compromised xApp cannot " lateral move" to unauthorized data or APIs.

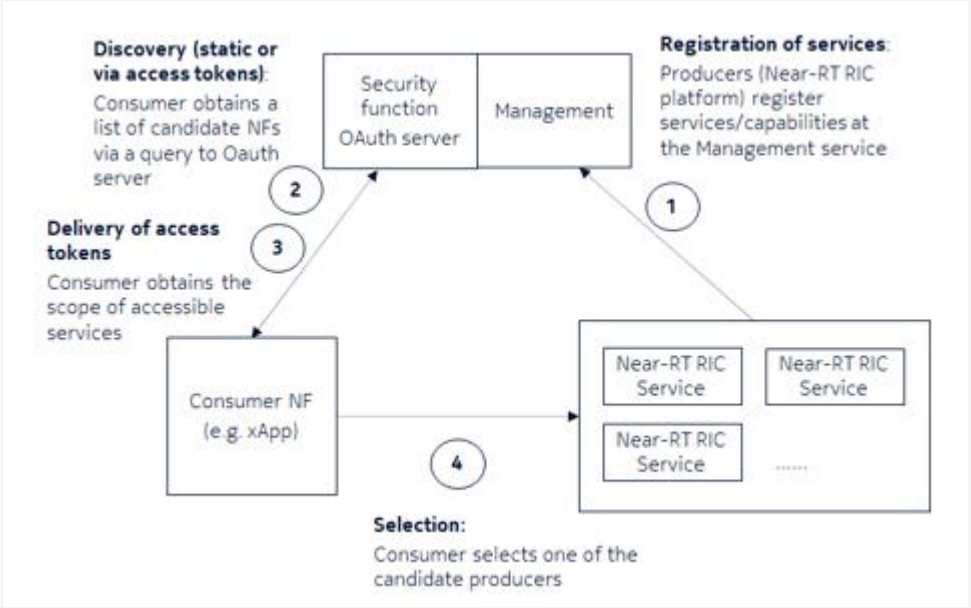


Figure: OAuth 2.0 Process in Near-RT RIC [2]

The Secure Near-RT RIC

The Near-RT RIC's open, disaggregated architecture provides immense power for RAN innovation. Securing this ecosystem requires a resilient RIC that can contain, verify, restrict and resolve.

By building these principles into the architecture and lifecycle management, the O-RAN ecosystem can harness the full potential of xApps while defending against the inherent threats of a multi-vendor, software-defined network.

References

1. O-RAN.WG3.TS.RICARCH-R004-v07.00 Technical Specification (O-RAN Work Group 3 (Near-Real-time RAN Intelligent Controller and E2 Interface) Near-RT RIC Architecture)
2. O-RAN.WG.11.TR.Security-Near-RT-RIC-xApps.0-R004-v06.00 Technical Report (O-RAN Work Group 11 (Security Work Group) Study on Security for Near Real Time RIC and xApps)
3. O-RAN.WG11.TR.Security-Near-RT-RIC-xApps.0-R004-v06.00 *(O-RAN Working Group 11 (Security WG) Technical Report on Security for Near-Real-Time RIC and xApps)*

THANK YOU!